Design and Analysis of Algorithms (21AD44)

LaTeX report

BS Keerthi(1NT21AD015)

Fathima Fahim(1NT21AD018)

N Suhaas Suran(1NT21AD034)

Shreya C(1NT21AD049)

Dept. of Artificial Intelligence and Data Science
Nitte Meenakshi Institute of Technology
Bengaluru - 560064.

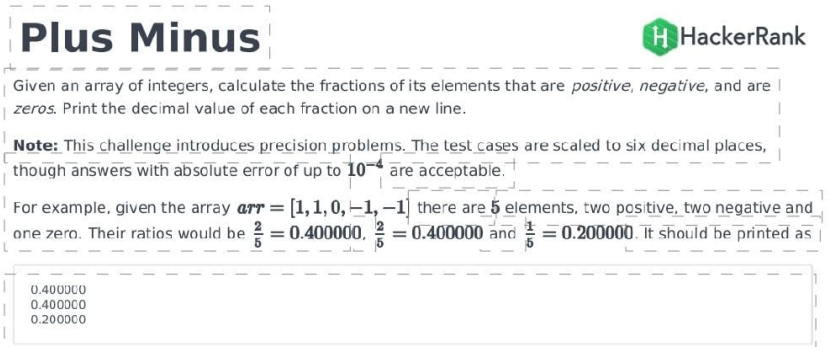
5 June, 2023

* Team Name : Digitalists

* Team Members : BS Keerthi, Fathima Fahim, N Suhaas Suran, Shreya C

* Problem number : 12

* Program Language : Python



The Plus Minus function, mentioned in the statement above, involves finding the ratio of positive, negative, and zero elements in an array of integers. The function calculates the proportion of each category by dividing the count of positive numbers, negative numbers, and zeros in the array by the total number of elements in the array.

Initial Implementation of the Plus Minus function:

1.Initialize three variables:These variables will keep track of the number of positive, negative, and zero elements in the array, respectively.

2.Iterate through each element in the array: The iteration will pass through element and the respective positive, negative and the zero factors will be incremented.

3.Calculate the ratios: Ratios of each factors is calculated for obtaining the final output.

4.Return the three ratios as a result.

This function is called by passing an array of integers, and it will returns the output containing the ratios of positive, negative, and zero elements in the array which is placed up to six decimal points.

0.1 Algorithm

```
defining function plusMinus(arr):
    Initialize variables pos, neu, and neg to 0

    for i from 0 to length of arr:
        if arr[i] > 0:
            Increment pos by 1
        elseif arr[i] < 0:
            Increment neg by 1
        else:
            Increment neu by 1

    Calculate the fraction of positive numbers (pos/length of arr)
    Calculate the fraction of negative numbers (neg/length of arr)
    Calculate the fraction of zero (neu/length of arr)

    Print the decimal representing the fraction of positive numbers
    Print the decimal representing the fraction of negative numbers
    Print the decimal representing the fraction of zero

Read an integer n from input, Read the array arr from input,
splitting the values and converting them to integers.

Call the plusMinus function with the array arr as input
```

0.2 Explanation

The algorithm defines a function called `plusMinus` that takes an array `arr` as input.

Inside the `plusMinus` function, three variables `pos`, `neu`, and `neg` are initialized to 0. These variables will keep track of the count of positive, zero, and negative numbers in the array.

The algorithm then iterates through each element in the array using a `for` loop. For each element, it checks its value:
- If the element is greater than 0, it increments the `pos` variable by 1.
- If the element is less than 0, it increments the `neg` variable by 1.
- If the element is equal to 0, it increments the `neu` variable by 1.

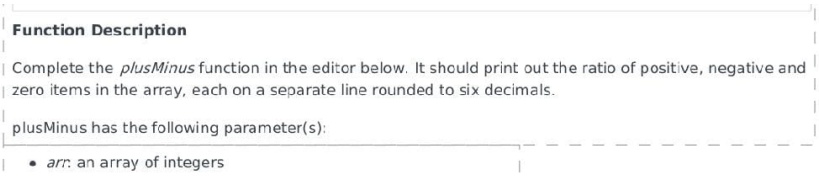
After counting the positive, zero, and negative numbers, the algorithm calculates the fractions by dividing the counts by the length of the array. The fractions are calculated as follows:
- Fraction of positive numbers: `pos/length of arr`
- Fraction of negative numbers: `neg/length of arr`
- Fraction of zero: `neu/length of arr`

Finally, the algorithm prints the decimal values representing the fractions of positive numbers, negative numbers and zero.

The code then reads an integer `n` from the input, followed by the array `arr` as space-separated integers. It calls the `plusMinus` function with `arr` as the input.

The time complexity of this algorithm is O(n), where n is the length of the input array, as it iterates through each element once. The space complexity is O(1) since the algorithm only uses a constant amount of additional space to store the counts and fractions.

0.3 Discussion on Data Structures and Efficiency


Function Description

Complete the *plusMinus* function in the editor below. It should print out the ratio of positive, negative and zero items in the array, each on a separate line rounded to six decimals.

plusMinus has the following parameter(s):

- *arr*: an array of integers

In the given code, the `plusMinus` function calculates the fractions of positive, negative, and zero numbers in the input array. Let's discuss the data structures used in the algorithm and their efficiency.

0.3.1 Variables

The algorithm utilizes three variables, `pos`, `neu`, and `neg`, to store the counts of positive, zero, and negative numbers, respectively. These variables are of type `integer` and require constant space. Since their space usage does not depend on the size of the input array, the space complexity of these variables is $O(1)$.

0.3.2 Array

The input array, `arr`, is represented as a one-dimensional array of integers. It serves as a container to store the input values and is traversed to count the number of positive, negative, and zero numbers. The array offers constant-time access to its elements, resulting in efficient random access. Therefore, the time complexity for accessing elements in the array is $O(1)$.

The algorithm does not employ advanced data structures like linked lists, trees, or hash tables. Instead, it relies on basic variables and arrays to store and process the data. This simplicity contributes to the algorithm's efficiency as it minimizes both time and space complexities.

The time complexity of the algorithm is $O(n)$, where n represents the length of the input array. This complexity arises from the linear traversal of each element in the array, performing constant-time operations. Hence, the time complexity scales linearly with the input size.

The space complexity of the algorithm is $O(1)$ since it only utilizes a fixed amount of additional space to store the count variables. This space requirement

remains constant, regardless of the input array's size.

In conclusion, the algorithm employs simple and efficient data structures, such as variables and arrays, to perform the required calculations on the input array. The algorithm's time and space complexities are optimal, making it an efficient solution for the given problem.

0.4 Time complexity

Time complexity is defined as the amount of time taken by an algorithm to run, as a function of the length of the input. It measures the time taken to execute each statement of code in an algorithm.
As in the algorithm Plus Minus problem takes linear time, or O(n) time, where 'n' is the size of array. The array size is from 0 to n-1. Informally, this means that the running time increases at most linearly with the size of the input(n).

More precisely, this means that there is a constant c such that the running time is at most **c(n)** for every input of size n. For example, a procedure that adds up all elements of a list requires time proportional to the length of the list, if the adding time is constant, or, at least, bounded by a constant.

The cases of Plus Minus are as follows:
• Best case: **Cbest(n)** – minimum number of times over inputs of size n
• Average case: **Cavg(n**) – "average" over inputs of size n
• Worst case: **Cworst(n)** – maximum number of times the basic operation is executed over inputs of size n.

0.4.1 CONSTRAINTS:

The constraints of Plus Minus statement are as follows:

Constraints

$$0 < n \leq 100$$
$$-100 \leq arr[i] \leq 100$$

Best Case:
1) $0 < n <= 100$:
The best case time complexity is O(1). We can simply check if the given value of n satisfies both conditions in a constant amount of time.
2) $-100 <= arr[i] <= 100$:
The best case time complexity is O(1) as well. We compare each element of the array 'arr' to verify if it falls within the specified range without iterating over the entire array.

Since both parts have a best case time complexity of O(1), the overall best case time complexity for the combined inequality is still O(1). This means that regardless of the size of the input or the number of elements in the array, the best case scenario allows for constant time determination of the truth of the inequalities.

Average Case:
Since the average case depends on the specific input distribution,we can make some assumptions to discuss an average case scenario.
1) $0 < n <= 100$:
In the average case, we can assume that n is equally likely to fall within any value between 1 and 100. Thus, on average, we would need to perform one comparison to determine if the condition is satisfied. Therefore, the time complexity for this part is O(1).
2) $-100 <= arr[i] <= 100$:
Assuming a uniform distribution, each element in the array 'arr' falls within the range of -100 to 100. In the average case, we would need to compare each element of the array to determine if it satisfies the condition. As a result, the average case time complexity for this part is O(n).

Since the first part has a time complexity of O(1) and the second part has a time complexity of O(n) in the average case, the overall average case time complexity for the combined inequality is O(n).

Worst Case:
1) $0 < n <= 100$:
The worst case scenario for this inequality would be when n takes on its largest possible value, which is 100. In this case, we would need to perform a single comparison to check if 0 ¡ 100, which takes constant time, denoted as O(1).
2) $-100 <= arr[i] <= 100$:
The worst case scenario for this inequality would be when we have an array, arr, with all its elements being the maximum possible value of 100 or the minimum possible value of -100. In this case, we would need to iterate through each element of the array and perform two comparisons for each element (one for $-100 <= arr[i]$ and another for $arr[i] <= 100$).

Therefore, Combining both parts, the overall worst case time complexity would be O(n) since the time complexity of the second part dominates when considering larger input sizes.

0.5 Execution of Code:

Complete plusMinus function is given below:

```
def plusMinus(arr):
    pos,neu,neg=0,0,0
    for i in range(0,len(arr)):
        if arr[i]>0:
            pos = pos + 1
        elif arr[i]<0:
            neg = neg + 1
        else:
            neu = neu + 1
    print ("The decimal representing the fraction of positive  numbers:%.6f" % (pos/len(arr)
    print ("The decimal representing the fraction of negative numbers:%.6f" %(neg/len(arr)))
    print ("The decimal representing the fraction of zero:%.6f" % (neu/len(arr)))

n = int(input())
arr = list(map(int, input().rstrip().split()))

plusMinus(arr)
```

0.6 Outputs of the Code

Output Format prints the following 3 lines:
1.The decimal representing the fraction of positive numbers in the array compared to its size.
2.The decimal representing the fraction of negative numbers in the array compared to its size.
3.The decimal representing the fraction of zeros in the array compared to its size.

Output 1:

```
suhaas_suran@ubuntu20:~/Desktop$ make run
python3 Team_17_A1.py
6
-4 3 -9 0 4 1
The decimal representing the fraction of negative numbers:0.333333
The decimal representing the fraction of zero:0.166667
The decimal representing the fraction of positive numbers:0.500000
```

Explanation: There are 3 positive, 2 negative and 1 zero in the array. The proportions of occurrence are 0.500000, 0.333333 and 0.166667 respectively.

Output 2:

```
suhaas_suran@ubuntu20:~/Desktop$ make run
python3 Team_17_A1.py
4
1 3 4 0
The decimal representing the fraction of negative numbers:0.000000
The decimal representing the fraction of zero:0.250000
The decimal representing the fraction of positive numbers:0.750000
```

Explanation: There are 4 positive, 0 negative and 1 zero in the array. The proportions of occurrence are 0.750000, 0.000000 and 0.250000 respectively.

Output 3:

```
suhaas_suran@ubuntu20:~/Desktop$ make run
python3 Team_17_A1.py
6
-9 -6 -8 -5 0 0
The decimal representing the fraction of negative numbers:0.666667
The decimal representing the fraction of zero:0.333333
The decimal representing the fraction of positive numbers:0.000000
```

Explanation: There are 0 positive, 4 negative and 2 zero in the array. The proportions of occurrence are 0.000000, 0.666667 and 0.333333 respectively.

Output 4:

```
suhaas_suran@ubuntu20:~/Desktop$ make run
python3 Team_17_A1.py
5
0 0 0 0 0
The decimal representing the fraction of negative numbers:0.000000
The decimal representing the fraction of zero:1.000000
The decimal representing the fraction of positive numbers:0.000000
```

Explanation: There are 0 positive, 0 negative and 5 zero in the array. The proportions of occurrence are 0.000000, 0.000000 and 1.000000 respectively.

Conclusion

This report gives us the information about the project worked on the given problem Statement "PlusMinus" choosing the Coding Language as "Python" under the course Design And Analysis of Algorithm. Source: Internet, Complexity Analysis, Introduction t0 Algorithm by Thomas H. Cormen, Lecture notes and Websites.